

Snort IDS + Wazuh SIEM Integration Lab Report

Network Intrusion Detection & Security Event Monitoring

Air University Kamra Campus | BS Cyber Security | May 2026

Lab Architecture

Component	VM	IP Address	Role
Kali Linux	Attacker VM	192.168.121.129	Launch attacks / simulate threat actor
Ubuntu Linux	Defender VM	192.168.121.132	Snort IDS + Wazuh Manager + Dashboard

Both VMs were connected using VMware NAT networking, which places them on the same internal subnet (192.168.121.0/24), allowing them to communicate directly with each other while still having internet access through the host machine.

Phase 1 — Network Setup & Connectivity Verification

What We Did:

Before installing any tools, we first confirmed that both VMs could communicate with each other. This is critical because Snort needs to actually see the attack traffic coming from Kali.

Why VMware NAT works here:

Unlike plain NAT which isolates VMs, VMware's NAT mode places all VMs on the same VMnet8 virtual switch. This means VMs can reach each other directly using their 192.168.121.x addresses.

Verification command:

```
ping 192.168.121.132 # From Kali to Ubuntu
```

Result: Successful ping replies confirmed both VMs are on the same network and can communicate.

Phase 2 — Snort IDS Installation & Configuration

What is Snort?

Snort is an open-source Network Intrusion Detection System (NIDS). It monitors network traffic in real time, analyzes packets against a set of rules, and generates alerts when suspicious activity is detected. It was originally created by Martin Roesch and is now maintained by Cisco.

Installation:

```
sudo apt update && sudo apt install snort -y
```

During installation, Snort asked for the HOME_NET value — this tells Snort which IP range is considered 'your network' (the network you want to protect). We set it to 192.168.0.0/16 to cover all possible NAT addresses.

Version Installed:

```
Snort Version: 2.9.20 GRE (Build 82)
```

Key configuration details:

```
Network Interface: ens33 | Ubuntu IP: 192.168.121.132
```

The network interface ens33 is the virtual NIC that Snort listens on. All traffic entering or leaving the VM through this interface will be inspected by Snort.

Phase 3 — Writing Custom Snort Detection Rules

What are Snort Rules?

Snort rules define what traffic patterns should trigger an alert. Each rule specifies the protocol, source/destination IP and port, and conditions to match. When a packet matches a rule, Snort generates an alert with the message defined in that rule.

```
Rules file location: /etc/snort/rules/local.rules
```

SID	Rule Name	Detects	How it works
1000001	ICMP Ping Detected	Ping / ICMP flood	Alerts on any ICMP packet to HOME_NET
1000002	Nmap SYN Scan	TCP port scanning	Detects 20+ SYN packets in 3 seconds from same source
1000003	Nmap Aggressive Scan	OS/service detection	Matches SYN+FIN flag combo used by Nmap -A
1000004	Nikto Web Scan	Web vulnerability scan	Looks for "Nikto" string in HTTP headers
1000005	SSH Brute Force	Password attacks on SSH	Detects 5+ SYN packets to port 22 in 10 seconds

Phase 4 — Wazuh SIEM Setup

What is Wazuh?

Wazuh is an open-source Security Information and Event Management (SIEM) platform. It collects logs from various sources, correlates security events, and presents them in a centralized dashboard. It is widely used in SOC (Security Operations Center) environments for threat detection, compliance monitoring, and incident response.

Wazuh Components Running on Ubuntu VM:

Component	Service Name	Purpose
Wazuh Manager	wazuh-manager	Core engine that processes and correlates alerts
Wazuh Indexer	wazuh-indexer	OpenSearch database that stores all alert data
Wazuh Dashboard	wazuh-dashboard	Web UI accessible at https://192.168.121.132

Issue Encountered: The Wazuh Indexer (OpenSearch) failed to start due to a timeout. This was resolved by running 'sudo systemctl daemon-reload' followed by restarting the wazuh-indexer service. OpenSearch is a Java-based application and sometimes needs a daemon reload after configuration changes on disk.

Phase 5 — Snort + Wazuh Integration

How the Integration Works:

Wazuh has a built-in log monitoring feature that can watch any file on the system. It also ships with pre-built decoders and rules for Snort's alert format. By pointing Wazuh at Snort's alert log file, every alert Snort generates gets automatically parsed, decoded, and indexed into the Wazuh dashboard as a security event.

Configuration added to `/var/ossec/etc/ossec.conf`:

```
<localfile>
  <log_format>snort-fast</log_format>
  <location>/var/log/snort/alert</location>
</localfile>
```

The log_format 'snort-fast' tells Wazuh to use its Snort-specific decoder, which knows how to parse Snort's alert format including timestamps, rule IDs, classifications, source/destination IPs, and protocol information.

Snort running as background daemon:

```
sudo snort -i ens33 -c /etc/snort/snort.conf -A fast -l /var/log/snort/ -D
```

-i ens33: Listen on interface ens33 | -c: Use main config file | -A fast: Write alerts in fast format | -l: Log directory | -D: Run as daemon (background)

Data Flow:

```
Kali VM (Attack) --> [Network Traffic] --> Snort IDS (Packet Inspection) -->
/var/log/snort/alert --> Wazuh Manager (Log Monitoring) --> OpenSearch Index --> Wazuh
Dashboard (Visualization)
```

Phase 6 — Attacks Launched from Kali Linux

The following attacks were launched from Kali Linux (192.168.121.129) targeting the Ubuntu VM (192.168.121.132). Each attack represents a common real-world threat scenario that a SOC analyst would encounter.

1. ICMP Flood

```
sudo ping -f 192.168.121.132
```

Sends a flood of ICMP Echo Request packets to overwhelm the target. This simulates a Denial of Service (DoS) attack. Snort captured over 262,000 ICMP packets during this test. In real environments, ICMP floods can saturate network bandwidth and disrupt services.

2. TCP SYN Scan (Nmap)

```
sudo nmap -sS 192.168.121.132
```

Sends SYN packets to every port without completing the TCP handshake (half-open scan). This is the most common port scanning technique used by attackers to discover open services without leaving a full connection log on the target.

3. Aggressive Scan (Nmap)

```
sudo nmap -A -T4 192.168.121.132
```

Performs OS detection, version detection, script scanning, and traceroute simultaneously. This gives the attacker a complete picture of the target system — what OS it runs, what services are open, and their versions. -T4 speeds up the scan timing.

4. Web Vulnerability Scan (Nikto)

```
nikto -h http://192.168.121.132
```

Scans the web server for known vulnerabilities, misconfigurations, dangerous files, and outdated software. Nikto sends thousands of requests with its signature in the HTTP User-Agent header, which Snort's rule detects by looking for the 'Nikto' string.

5. SSH Brute Force (Hydra)

```
hydra -l root -P /usr/share/wordlists/rockyou.txt 192.168.121.132 ssh -t 4
```

Attempts to log into SSH by trying thousands of passwords from the rockyou.txt wordlist. This simulates a credential stuffing / brute force attack. Snort detects the rapid succession of SYN packets to port 22 from the same source IP.

6. UDP Flood (hping3)

```
sudo hping3 --udp -p 53 --flood 192.168.121.132
```

Floods the target with UDP packets on port 53 (DNS port). UDP floods are used in DDoS attacks because UDP is connectionless and easier to spoof. Targeting port 53 simulates a DNS amplification attack pattern.

7. TCP SYN Flood (hping3)

```
sudo hping3 -S --flood -p 80 192.168.121.132
```

A low-level SYN flood targeting port 80 (HTTP). Unlike Nmap's scan, this is purely a DoS attack — it sends SYN packets without waiting for responses, exhausting the target's TCP connection table and potentially crashing web services.

8. Xmas Scan (Nmap)

```
sudo nmap -sX 192.168.121.132
```

Sets the FIN, PSH, and URG TCP flags simultaneously — 'lighting up the packet like a Christmas tree'. This unusual flag combination can bypass some simple firewalls and is used to fingerprint operating systems based on their response behavior.

9. FIN Scan (Nmap)

```
sudo nmap -sF 192.168.121.132
```

Sends packets with only the FIN flag set. On closed ports, the target responds with RST. On open ports, the packet is ignored. This can evade stateless packet filters that only block SYN packets, making it a stealthier reconnaissance technique.

10. NULL Scan (Nmap)

```
sudo nmap -sN 192.168.121.132
```

Sends packets with no TCP flags set at all. Like FIN and Xmas scans, this exploits RFC 793 behavior differences between open and closed ports. It is very stealthy and can bypass many firewall rules that only inspect standard flag combinations.

Phase 7 — Wazuh Dashboard Results

Alert Classification in Wazuh:

Wazuh processed Snort alerts and classified them using its built-in IDS ruleset. The alerts appeared in Security Events with the following classifications:

Rule ID	Description	Level	Meaning
20100	First time this IDS alert is generated	8	New/unique attack signature seen for first time
20101	IDS event	6	Recurring Snort alert matching known pattern

How to view Snort alerts on Wazuh Dashboard:

Navigate to: Security Events > Search 'ids' in the search bar > Set time range to Last 15 minutes > Click Refresh. Alerts will show as 'IDS event' entries with source IP, destination IP, and the specific Snort rule that triggered.

Key Observation:

The integration successfully demonstrated real-time threat detection: attacks launched from Kali appeared as IDS alerts in the Wazuh dashboard within seconds. The total alert count increased from 25 to 33+ during testing, with alerts flowing in real time. A PDF report was also generated directly from the Wazuh dashboard under Management > Reporting.

Key Configuration Files

File Path	Purpose
/etc/snort/rules/local.rules	Custom Snort detection rules written for this lab
/etc/snort/snort.conf	Main Snort configuration file (HOME_NET, rule paths, output)
/var/ossec/etc/ossec.conf	Wazuh config — added localfile block to monitor Snort logs
/var/log/snort/alert	Snort alert output file in fast format (monitored by Wazuh)

Conclusion

This lab successfully demonstrated a complete Network Intrusion Detection and SIEM integration pipeline using industry-standard open-source tools. The setup mirrors real-world SOC environments where Snort (or Suricata) feeds alerts into a SIEM platform like Wazuh, Splunk, or ELK Stack for centralized monitoring and analysis.

Key skills demonstrated in this lab include: network traffic analysis, IDS rule writing, SIEM log integration, attack simulation, and alert triage — all core competencies required for SOC Analyst and Penetration Testing roles.